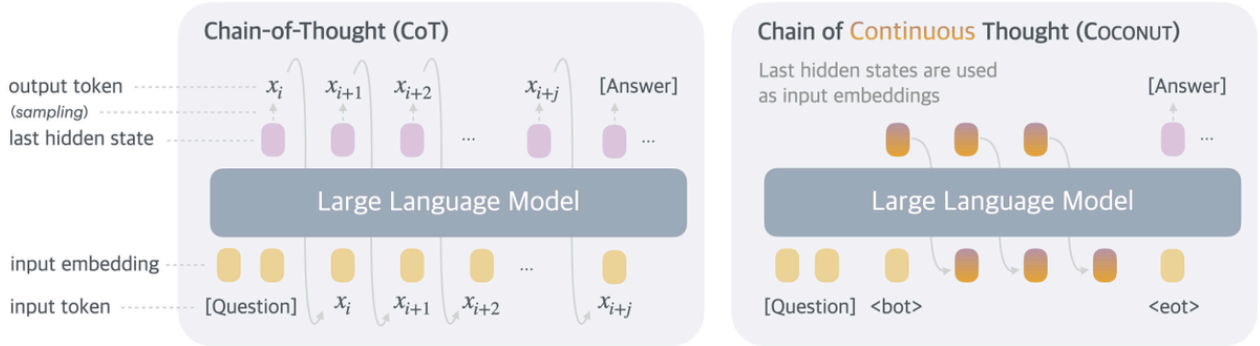
Training Large Language Models to Reason in a Continuous Latent Space

> #Inference #Finetuning



- What problem does this framework solve?
  - **Improper allocation of computes during inference**: Models tend to allocate the same amount of computes to the tokens that keep the fluency of the response instead of with an actual meaning
  - **Determine the next token too early**: The autoaggressive nature of LLMs tends to make them determine the output token too early (greedy pattern), which may not be the optimal, and stuck into a less optimal response.
- What does this framework do?
  - Normally, the LLM does the following operation:

    $$H_t = \text{Transformer}(\text{Emb}(x_t))$$
    $$M(x_{t+1}|x_{\leq t}) = \text{Softmax}(WH_t)$$

    where in the 2nd step, the token is determined.
  - This framework proposes to introduce another mode of generation: **The Latent Mode**.
  - Assume that the latent mode happens, the model will no longer translate the hidden state to logits, calculate softmax, output the token, and add it back to the sequence; instead, we directly add hidden state (the latent representation) to the context and use it as an input.
  - Multi-curriculum Training
    - Stage 0: Like normal SFT, finetune the model with CoT datasets.
    - Stage k: Remove the 1st k CoT sections from the train data and replace each CoT section with c hidden states.
    - The loss (negative log-likelihood) is only calculated after the model is switched to language mode, ignoring the latent mode and the question. *We are result-oriented and don't want the latent mode to "memorize" and "compress" the removed CoT steps; we want it to predict the final result.*
  - Inference Stage: When the model receives the input, it will start the latent mode by inserting a special token `<bot>`. When the latent mode's length reaches a threshold, it will quit the mode by using `<eot>` token.
  - **Special Findings**: The Generation Tree

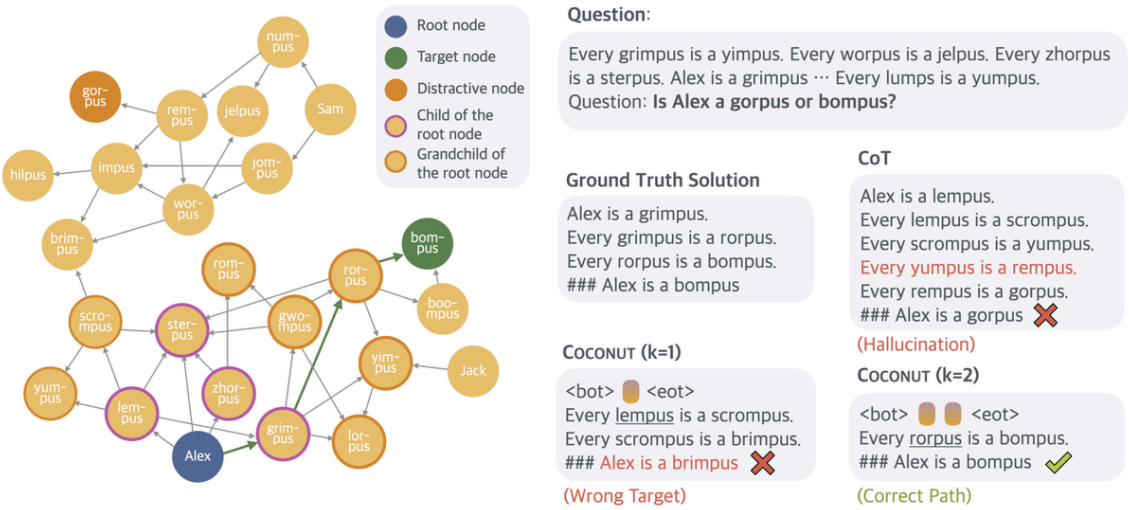


Figure 4 A case study of ProsQA. The model trained with *CoT* hallucinates an edge (*Every yumpus is a rempus*) after getting stuck in a dead end. COCONUT (k=1) outputs a path that ends with an irrelevant node. COCONUT (k=2) solves the problem correctly.

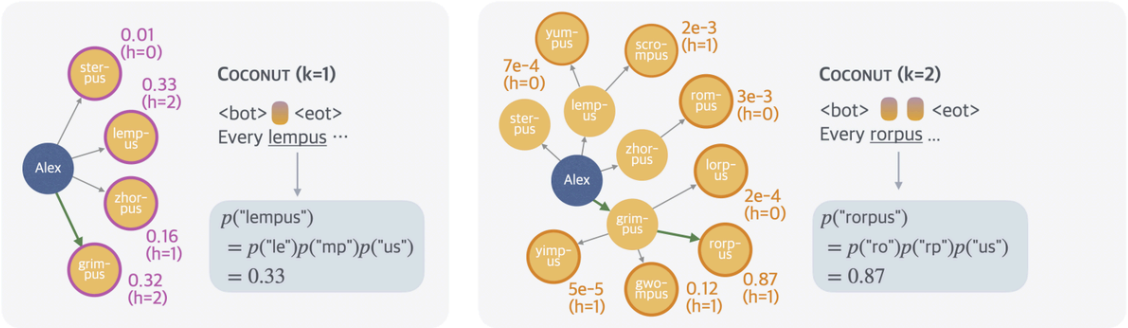


Figure 5 An illustration of the latent search trees. The example is the same test case as in Figure 4. The height of a node (denoted as h in the figure) is defined as the longest distance to any leaf nodes in the graph. We show the probability of the first concept predicted by the model following latent thoughts (e.g., "lempus" in the left figure). It is calculated as the multiplication of the probability of all tokens within the concept conditioned on previous context (omitted in the figure for brevity). This metric can be interpreted as an implicit value function estimated by the model, assessing the potential of each node leading to the correct answer.

- Researchers probed a traditional CoT LLM and this framework to study the pattern of the generation.
- The normal CoT LLM runs like a greedy search: it will anchor each step at the token with the highest possibility and go on in the same pattern.
    - However, it is found that the model is usually not "confident" at the very beginning of the inference. That is to say, even if one token wins the highest probability score, it is not always the right path (*This is a possible source of the hallucination of LLMs*).
- This framework, on the other hand, runs like a BFS. Even if the model gives a high score to a token in the 1st token prediction, it may give a token the highest score in the 2nd prediction, and this token can be from another branch (*because it does not decode the hidden state to determine the token, which makes it more agile*).

- Experiment Results
    - Superior logical reasoning performance: On complex reasoning tasks requiring extensive planning and search (such as ProntoQA and the newly proposed ProsQA), COCONUT achieves higher accuracy than traditional language CoT methods.
    - Enhanced efficiency: COCONUT generates significantly fewer discrete tokens during inference, striking an optimal balance between accuracy and computational efficiency.

    - Reduced Hallucinations: The continuous space's fault tolerance and parallel exploration capabilities substantially mitigate “hallucinations” and erroneous target generation caused by early decision errors.
- Work Cited
    - Paper
    - Projects
    - Chat History
    - NotebookLM